Theoretical Guide
py.Sandu

Élvis Miranda, Leonardo Krauss & Rafael Alencar

1 C++

1.1 System optimization

For faster IO, we can use the following code:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
cout.tie(nullptr);
```

This code disables the synchronization between C++ streams and C streams, which can significantly improve the performance of input and output operations. Additionally, it unties the input and output streams, allowing them to operate independently, further enhancing the speed of IO operations.

1.2 Containers

- **std::vector:** A dynamic array. Most methods give O(1) time complexity for access and O(N) for insertion/deletion (except at the end, which is O(1)).
- **std::list:** A doubly linked list. Most methods give O(1) time complexity for insertion/deletion (given an iterator) and O(N) for access.
- **std::deque:** A double-ended queue. Most methods give O(1) time complexity for access and O(1) for insertion/deletion at both ends, but O(N) for insertion/deletion in the middle.
- **std::stack:** A container adapter that gives the functionality of a stack (LIFO). Most methods give O(1) time complexity for push, pop, and top operations.
- **std::queue:** A container adapter that gives the functionality of a queue (FIFO). Most methods give O(1) time complexity for push, pop, and front/back operations.
- **std::priority_queue:** A container adapter that gives the functionality of a priority queue. Most methods give O(log N) time complexity for push and pop operations, and O(1) for top operation.
- **std::set:** A sorted associative container that contains unique elements. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::map:** A sorted associative container that contains key-value pairs with unique keys. Most methods give O(log N) time complexity for insertion, deletion, and search operations.
- **std::unordered_set:** An unordered associative container that contains unique elements. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.
- **std::unordered_map:** An unordered associative container that contains key-value pairs with unique keys. Most methods give O(1) average time complexity for insertion, deletion, and search operations, but O(N) in the worst case.

1.3 Algorithms

- **std::sort:** The `std::sort` function in C++ is typically implemented using a hybrid sorting algorithm called Introsort, which combines Quick Sort, Heap Sort, and Insertion Sort. It has an average time complexity of O(N log N).
- **std::lower_bound and std::upper_bound:** These functions are used to find the lower and upper bounds of a value in a sorted range. They have a time complexity of O(log N) since they use binary search.
- **std::next_permutation:** This function generates the next lexicographical permutation of a sequence. It is useful for generating all permutations of a sequence in sorted order. The time complexity is O(N) for each call, where N is the length of the sequence.

2 Number Theory

2.1 Modular Arithmetic & Number Theory

- **Properties of Modulo:**

- Sum: $(a+b)\%m = (a\%m + b\%m)\%m$
- Subtraction: $(a-b)\%m = (a\%m - b\%m + m)\%m$
- Product: $(a \times b)\%m = (a\%m \times b\%m)\%m$

- **Modular Division:** $(a/b) \equiv (a \times b^{-1}) \pmod{m}$, where $b \times b^{-1} \equiv 1 \pmod{m}$. Valid if $\gcd(b, m) = 1$.
- **Euler's Totient Theorem:** If $\gcd(a, m) = 1$, then $a^{\phi(m)} \equiv 1 \pmod{m}$.
- **Fermat's Little Theorem:** If p is prime, $a^{p-1} \equiv 1 \pmod{p}$. Inverse: $a^{-1} \equiv a^{p-2} \pmod{p}$.
- **Power Reduction:** If p is prime: $a^b \equiv a^{b \pmod{p-1}} \pmod{p}$. General: $a^b \equiv a^{b \pmod{\phi(m)}} \pmod{m}$ (if $\gcd(a, m) = 1$).
- **Bézout's Identity:** For a, b, there exist x, y such that $ax + by = \gcd(a, b)$.

2.2 Divisors and Prime Factorization

For an integer $n > 1$, let its prime factorization be $n = p_1^{a_1} \cdot p_2^{a_2} \dots p_k^{a_k}$.

- **Number of Divisors (τ):** The total count of divisors of n.

$$\tau(n) = \prod_{i=1}^{k} (a_i + 1)$$

- **Sum of Divisors (σ):** The sum of all positive divisors of n.

$$\sigma(n) = \prod_{i=1}^{k} \left(\frac{p_i^{a_i+1} - 1}{p_i - 1} \right)$$

- **Product of Divisors (P):** The product of all divisors of n.

$$P(n) = n^{\frac{\tau(n)}{2}}$$

  Note: If n is a perfect square, $\tau(n)$ is odd, but the result remains an integer.

- **Euler's Totient Function (ϕ):** Number of integers $k \in [1, n]$ such that $\gcd(k, n) = 1$.

$$\phi(n) = n \prod_{i=1}^{k} \left(1 - \frac{1}{p_i} \right)$$

3 Constants

3.1 Time and memory limits

- **1 second limit:** 1 second is usually enough for a program to perform around 10^8 operations for O(N) algorithms and around 10^6 operations for O(N log N) algorithms.
- **256Mb limit:** 256 megabytes is usually enough to store around 10^7 integers (since each integer typically takes 4 bytes).

3.2 Mathematical

- **Pi (π)**: Usually defined as acos(-1.0), approximately equal to 3.14159. In C++ 20, it is available as `std::numbers::pi`.
- **Euler (e)**: Usually defined as the base of the natural logarithm, approximately equal to 2.71828. In C++ 20, it is available as `std::numbers::e`.
- **Golden Ratio (ϕ)**: Usually defined as (1 + sqrt(5)) / 2, approximately equal to 1.61803. In C++ 20, it is available as `std::numbers::phi`.

3.3 Data type limits

- **Integer limits**: The typical 32 bit signed integer bounds are ($\approx \pm 2 \times 10^9$) and is obtained by using the `INT_MAX` and `INT_MIN` constants, while the 64 bit signed integer bounds are ($\approx \pm 9 \times 10^{18}$) and is obtained by using the `LLONG_MAX` and `LLONG_MIN` constants.
- **Floating-point limits**: The typical 32 bit floating-point bounds are approximately $\pm 3.4 \times 10^{38}$ and is obtained by using the `FLT_MAX` and `FLT_MIN` constants, while the 64 bit floating-point bounds are approximately $\pm 1.7 \times 10^{308}$ and is obtained by using the `DBL_MAX` and `DBL_MIN` constants.

4 Math

4.1 System Solving

- **Definition:** A **system of equations** is a set of two or more equations involving the same set of variables. A solution to the system is a set of values for the variables that satisfies all the equations in the system.
- **Methods of Solving:**

- **Substitution:** Solve one equation for one variable and substitute that expression into the other equation(s).
- **Elimination:** Add or subtract the equations to eliminate one variable, making it easier to solve for the remaining variable(s).
- **Graphical:** Graph the equations on a coordinate plane and identify the point(s) of intersection, which represent the solution(s) to the system.
- **Matrix Method:** Use matrices and row operations to solve systems of linear equations, particularly useful for larger systems.

- **Types of Solutions:**
  - **Unique Solution:** The system has exactly one solution, meaning the equations represent lines (in 2D) or planes (in 3D) that intersect at a single point.
  - **Infinite Solutions:** The system has infinitely many solutions, meaning the equations represent the same line (in 2D) or plane (in 3D).
  - **No Solution:** The system has no solution, meaning the equations represent parallel lines (in 2D) or planes (in 3D) that do not intersect.
- **Algorithmic Approach:**
  - For linear systems, use Gaussian elimination or LU decomposition to find solutions efficiently.
  - For nonlinear systems, numerical methods such as Newton's method or fixed-point iteration can be employed to approximate solutions.

4.2 Polynomials

- **Definition:** A **polynomial** is an expression of the form

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0,$$

  where $a_n, a_{n-1}, \ldots, a_0$ are coefficients and n is a non-negative integer called the degree of the polynomial.
- **Root of a polynomial:** A **root** of a polynomial $P(x)$ is a value r such that $P(r) = 0$.
- **Fundamental Theorem of Algebra:** Every non-constant polynomial with complex coefficients has at least one complex root.
- **Factorization:** A polynomial of degree n can be factored into linear factors over the complex numbers, i.e., if $P(x)$ is a polynomial of degree n, then it can be expressed as

$$P(x) = a_n(x - r_1)(x - r_2) \cdots (x - r_n),$$

  where $r_1, r_2, \ldots, r_n$ are the roots of the polynomial (counting multiplicities).
- **Inversion:** Given a polynomial $P(x)$, the **inverse** of $P(x)$, denoted as $P^{-1}(x)$, is a polynomial such that $P(P^{-1}(x)) = x$ for all x in the domain of P^{-1}.

4.3 Algorithms

- **Fast Fourrier Transform (FFT):** An efficient algorithm to compute the discrete Fourier transform (DFT) and its inverse, which can be used for polynomial multiplication.
- **Lagrange Inversion Formula:** A formula that provides the coefficients of the inverse of a power series, which can be applied to polynomials.
- **Lagrange Interpolation:** A method for finding a polynomial that passes through a given set of points.
- **Newton's Method:** An iterative method for finding successively better approximations to the roots of a real-valued function, which can be applied to polynomials.

4.4 Numerical approximation

- **Definition:** A numerical approximation is a method of estimating the value of a mathematical expression or function using numerical techniques. It is often used when an exact solution is difficult or impossible to obtain.
- **Types of numerical approximation:**
  - **Taylor series approximation:** This method uses the derivatives of a function at a specific point to create a polynomial that approximates the function near that point.
  - **Finite difference approximation:** This method approximates the derivative of a function by using the values of the function at nearby points (ex: Newton's method or fixed-point iteration).

- **Numerical integration:** This method approximates the value of an integral by using numerical techniques such as the trapezoidal rule or Simpson's rule.
- **Interpolation:** This method estimates the value of a function at a point by using the values of the function at nearby points. Common interpolation methods include linear interpolation and polynomial interpolation.
- **Euler's method:** This is a numerical method for solving ordinary differential equations (ODEs). It approximates the solution by using the derivative of the function at a specific point to estimate the value of the function at the next point.

4.5 Numerical Algorithms

- **Newton's Method:** This is an iterative method for finding the roots of a function. It uses the derivative of the function to iteratively improve the approximation of the root. The formula for Newton's method is given by:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

where x_n is the current approximation, $f(x_n)$ is the value of the function at x_n, and $f'(x_n)$ is the derivative of the function at x_n.

- **Fixed-point Iteration:** This method is used to find a fixed point of a function, which is a point where the function equals its own value. The iteration formula is given by:

$$x_{n+1} = g(x_n)$$

where g is a function that transforms the original problem into a fixed-point problem. The convergence of this method depends on the properties of the function g.

- **Simpson's Rule:** This is a numerical method for approximating the value of a definite integral. It uses parabolic segments to approximate the area under the curve. The formula for Simpson's rule is given by:

$$\int_a^b f(x)\,dx \approx \frac{b-a}{6}[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)]$$

where a and b are the limits of integration, and f is the function being integrated.

- **Trapezoidal Rule:** This is another numerical method for approximating the value of a definite integral. It uses trapezoids to approximate the area under the curve. The formula for the trapezoidal rule is given by:

$$\int_a^b f(x)\,dx \approx \frac{b-a}{2}[f(a) + f(b)]$$

where a and b are the limits of integration, and f is the function being integrated.

- **Linear Interpolation:** This method estimates the value of a function at a point by using the values of the function at two nearby points. The formula for linear interpolation is given by:

$$f(x) \approx f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0}(x - x_0)$$

where x_0 and x_1 are the two nearby points, and $f(x_0)$ and $f(x_1)$ are the values of the function at those points. This method is simple and efficient for approximating values within the range of the two points.

- **Polynomial Interpolation:** This method estimates the value of a function at a point by using the values of the function at multiple nearby points to construct a polynomial that approximates the function. The most common form of polynomial interpolation is Lagrange interpolation, which constructs a polynomial that passes through all the given points. The formula for Lagrange interpolation is given by:

$$P(x) = \sum_{i=0}^{n} f(x_i) \prod_{j=0, j\neq i}^{n} \frac{x - x_j}{x_i - x_j}$$

where $x_0, x_1, \ldots, x_n$ are the given points, and $f(x_i)$ are the values of the function at those points. This method can provide a good approximation for smooth functions but may suffer from Runge's phenomenon if the degree of the polynomial is too high or if the points are not chosen carefully.

- **Euler's Method:** This is a numerical method for solving ordinary differential equations (ODEs). It approximates the solution by using the derivative of the function at a specific point to estimate the value of the function at the next point. The formula for Euler's method is given by:

$$y_{n+1} = y_n + f(t_n, y_n)\Delta t$$

where y_n is the current approximation of the solution, $f(t_n, y_n)$ is the value of the derivative at the current point, and Δt is the step size. This method is simple and easy to implement but may not be very accurate for stiff equations or when a small step size is required.

5 Notes

6 Identities

7 Geometry

7.1 Lines & Polygons

- **Slope-Intercept:** $y = mx + c$
- **General Line Equation:** $Ax + By + C = 0$
- **Distance from point** (x_0, y_0) **to line** $Ax + By + C = 0$**:** $d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$
- **Shoelace Formula (Area of Polygon):** $A = \frac{1}{2}|\sum_{i=1}^{n-1}(x_i y_{i+1} - x_{i+1} y_i) + (x_n y_1 - x_1 y_n)|$
  OBS: All vertices must be in order, either clockwise or counterclockwise
- **Pick's Theorem (Area of Lattice Polygons):** $A = I + \frac{B}{2} - 1$, where I is interior points and B is boundary points

7.2 Circles

- **Area:** πr^2
- **Circumference:** $2\pi r$
- **Equation:** $(x - h)^2 + (y - k)^2 = r^2$

7.3 Basic 2D Geometry

- **Distance Formula:** $\sqrt{\Delta x^2 + \Delta y^2}$
- **Vector Definition:** $\vec{v} = (x, y)$
- **Dot Product** $(\vec{a} \cdot \vec{b})$**:** $x_1 x_2 + y_1 y_2 = |\vec{a}||\vec{b}| \cos\theta$
  Usage: Finding angles, checking perpendicularity $(\vec{a} \cdot \vec{b} = 0)$
- **Cross Product** $(\vec{a} \times \vec{b})$:$x_1 y_2 - y_1 x_2 = |\vec{a}||\vec{b}| \sin\theta$
  Usage: 2D orientatiom, calculating signed area, collinearity $(\vec{a} \times \vec{b} = 0)$
- **Midpoint:** $M = (\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2})$

7.4 Algorithms/Concepts

- **Convex Hull (Monotone Chain or Graham Scan):** Finds the smallest convex polygon containging all points
- **Line Segment Intersection:** Using cross products to check if two segments intersect
- **Point in Polygon:** Using ray casting (winding number) algorithm for O(n) complexity or BSP Tree for O(log n) complexity
- **Angle of vector:** Use `atan2(y,x)` to find the angle of a vector.

8 Bitwise

8.1 Techniques

- **Check Even/Odd**: $n\&1$ can be used to check if a number is even (result is 0) or odd (result is 1).
- **Power of 2 check:** $(n\&\&!(n\&(n-1)))$ returns true if n is a power of 2.
- **Count 1s:** To count the number of 1s in the binary representation of a number, you can use the expression $n\&(n-1)$ repeatedly until n becomes 0. Each time you perform this operation, it removes the rightmost 1 from the binary representation, allowing you to count how many times you can do this before n becomes 0.
- **Isolate/Clear Rightmost Set Bit:** To isolate the rightmost set bit of a number, you can use the expression $n\&(-n)$. To clear the rightmost set bit, you can use $n\&(n-1)$.
- **Subset Generation:** Use a loop from 0 to $2^n - 1$ to generate all subsets of a set of size n. Each number in this range represents a subset, where the binary representation indicates which elements are included 1 or excluded 0.

8.2 Algebraic Properties

- **Addition Relationship:** $a + b = (a \oplus b) + 2 \cdot (a \wedge b)$

- **XOR Relationship:** $a|b = (a \oplus b) + (a \wedge b)$
- **AND Relationship:** $a \wedge b = (a + b) - (a \oplus b)$
- **Range XOR:** The XOR of all numbers from 1 to n follows a pattern based on $n \pmod 4$:
  - If $n \mod 4 = 0$, then XOR sum $= n$
  - If $n \mod 4 = 1$, then XOR sum $= 1$
  - If $n \mod 4 = 2$, then XOR sum $= n + 1$
  - If $n \mod 4 = 3$, then XOR sum $= 0$

9 Counting Problems

9.1 Permutation

- **Definition:** A permutation of a set is an arrangement of its members into a sequence or linear order. The number of permutations of n distinct objects is given by $n!$ (n factorial).
- **Permutations with Repetition:** If there are n objects where there are n_1 indistinguishable objects of one type, n_2 indistinguishable objects of another type, and so on, the number of distinct permutations is given by $\frac{n!}{n_1! \cdot n_2! \cdot \ldots}$.
- **Circular Permutations:** The number of ways to arrange n distinct objects in a circle is given by $(n-1)!$, since one object can be fixed to break the circular symmetry.

9.2 Modular Inverse Factorial

1. Calculate $n! \pmod p$ for a given prime p
2. Compute the modular inverse of $n!$ using Fermat's Little Theorem, which states that if p is a prime and a is an integer not divisible by p, then $a^{p-1} \equiv 1 \pmod p$. Therefore, the modular inverse of $n!$ can be calculated as $(n!)^{p-2} \pmod p$.
3. Use the precomputed factorial values and their inverses to efficiently compute the modular inverse of $n!$ for multiple queries.

9.3 Combination

- **Definition:** A combination is a selection of items from a larger set, where the order of selection does not matter. The number of combinations of n distinct objects taken r at a time is given by the formula:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

- **Combinations with Repetition:** If repetition of items is allowed, the number of combinations of n distinct objects taken r at a time is given by the formula:

$$C(n + r - 1, r) = \frac{(n + r - 1)!}{r!(n-1)!}$$

- **Pascal's Triangle:** The number of combinations can also be found using Pascal's Triangle, where the entry in the n-th row and r-th column corresponds to $C(n, r)$.

10 Progressions

A progression is a sequence of numbers ordered according to a specific, logical rule or formula, where each term is derived from the previous one.

10.1 Special Sums

- Sum of first n natural numbers: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
- Sum of first n odd numbers: $\sum_{i=1}^{n} (2i - 1) = n^2$
- Sum of first n even numbers: $\sum_{i=1}^{n} 2i = n(n + 1)$
- Sum of first n squares: $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
- Sum of first n cubes: $\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$
- Sum of first n squares of odd numbers: $\sum_{i=1}^{n} (2i - 1)^2 = \frac{n(4n^2-1)}{3}$
- Sum of first n squares of even numbers: $\sum_{i=1}^{n} (2i)^2 = \frac{4n(n+1)(2n+1)}{6}$

10.2 Geometric Progression

General Term: $a_1 \times q^{n-1}$

Sum:

$$\sum_{i=1}^{n} a_1 q^{i-1} = a_1 \frac{1-q^n}{1-q}$$

Infinite Sum:

$$-1 < q < 1$$

$$\frac{a_1}{1-q}$$

10.3 Arithmetic Progression

General Term: $a_1 + (n-1)r$

Sum:

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$